



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Diseño y Programación de Bases de Datos GT1	Unidad #: 3	Tema: Manejo de seguridades e introducción al PL /SQL
	Fecha:	Tutor: Ing. Gabriel Enrique Carballo Portillo

Introducción:

Bienvenidos al material de lectura 3 de la Unidad #3 de la materia Diseño y Programación de Bases de Datos .

El uso avanzado de cursores en PL/SQL permite una gestión más precisa y dinámica de consultas SQL, facilitando tareas como la personalización de resultados mediante alias en columnas, el filtrado con parámetros al momento de abrir un cursor, y el manejo de errores a través de cursores implícitos. Estas herramientas proporcionan mayor flexibilidad y control sobre el tratamiento de los datos en bloques PL/SQL.

Además, los cursores pueden utilizarse para actualizar registros directamente desde los resultados de una consulta mediante la cláusula FOR UPDATE, lo que permite bloquear y modificar filas de manera eficiente. También se puede emplear el identificador ROWID para actualizar registros específicos, evitando bloqueos extensivos sobre los datos y mejorando el rendimiento en ciertas operaciones críticas.

[illegible]

Los parámetros formales de un cursor son parámetros de entrada. El ámbito de estos parámetros es local al cursor, por eso solamente pueden ser referenciados dentro de la consulta.

DECLARE

```
...  
CURSOR cur1  
(v_departamento NUMBER,  
v_oficio VARCHAR2 DEFAULT 'DIRECTOR')  
IS SELECT apellido, salario FROM emple  
WHERE dept_no = v_departamento AND oficio = v_oficio;
```

La apertura del cursor pasándole parámetros se hará:

```
OPEN nombrecursor [ ( parámetro1, parámetro2, ...) ];
```

Donde parámetro1, parámetro2, ... son expresiones que contienen los valores que se pasarán al cursor. No tienen que ser los mismos nombres de las variables indicadas como parámetros al declarar el cursor; es más, si lo fueran, serían consideradas como variables distintas.

Supongamos, por ejemplo, las siguientes declaraciones:

DECLARE

```
v_dep emple.dept_no%TYPE;  
v_ofi emple.oficio%TYPE;  
CURSOR cur1  
(v_departamento NUMBER,  
v_oficio VARCHAR2 DEFAULT 'DIRECTOR')  
IS SELECT apellido, salario FROM emple  
WHERE dept_no = v_departamento AND oficio = v_oficio;  
...
```

Cualquiera de los siguientes comandos abrirá el cursor:

BEGIN

```
...  
OPEN cur1(v_dep);  
OPEN cur1(v_dep, v_ofi);  
OPEN cur1(20, 'VENDEDOR');
```

...
Debemos recordar que:

- Los parámetros formales de un cursor son siempre IN y no devuelven ningún valor ni pueden afectar a los parámetros actuales.

- La recogida de datos se hará, igual que en otros cursores explícitos, con FETCH.
- La cláusula WHERE asociada al cursor se evalúa solamente en el momento de abrir el cursor. En ese momento es cuando se sustituyen las variables por su valor.

En el caso de los cursores FOR...LOOP, puesto que la instrucción OPEN va implícita, el paso de parámetros se hará a continuación del identificador del cursor en la instrucción FOR...LOOP, tal como se muestra en el ejemplo:

```
...  
FOR reg_emple IN cur1(20, 'DIRECTOR') LOOP  
...
```

3.10.7 Atributos en cursores implícitos

Oracle abre implícitamente un cursor cuando procesa un comando SQL que no esté asociado a un cursor explícito. El cursor implícito se llama SQL y dispone también de los cuatro atributos mencionados, que pueden facilitarnos información sobre la ejecución de los comandos SELECT INTO, INSERT, UPDATE y DELETE.

El valor de los atributos del cursor SQL se refiere, en cada momento, a la última orden SQL:

- 1 **SQL%NOTFOUND** dará TRUE si el último INSERT, UPDATE, DELETE o SELECT INTO ha fallado (no ha afectado a ninguna fila).
- 2 **SQL%FOUND** dará TRUE si el último INSERT, UPDATE, DELETE o SELECT INTO ha afectado a una o más filas.
- 3 **SQL%ROWCOUNT** devuelve el número de filas afectadas por el último INSERT, UPDATE, DELETE o SELECT INTO.
- 4 **SQL%ISOPEN** siempre devolverá FALSO, ya que ORACLE cierra automáticamente el cursor después de cada orden SQL.

Es preciso hacer algunas observaciones respecto al uso de atributos en cursores implícitos pues su comportamiento difiere, en algunos casos, al de los cursores explícitos.

De hecho, como acabamos de ver, el cursor estará cerrado inmediatamente después de procesar la orden SELECT...INTO, que es cuando se pregunta por su atributo o atributos (lo cual no determina un error como pasaba en los cursores explícitos).

A continuación, se especifican algunas peculiaridades en el comportamiento y uso de los atributos en cursores implícitos:

- Devolverán un valor relativo a la última orden INSERT, UPDATE, DELETE o

SELECT...INTO, aunque el cursor esté cerrado.

- En el caso de que se trate de un SELECT...INTO, debemos tener en cuenta que ha de devolver una fila y sólo una, pues de lo contrario se producirá un error y se levantará automáticamente una excepción:
 - NO_DATA_FOUND, si la consulta no devuelve ninguna fila.
 - TOO_MANY_ROWS, si la consulta devuelve más de una fila.

Se detendrá la ejecución normal del programa y bifurcará a la sección EXCEPTION. Por tanto, cualquier comprobación de la situación del cursor en esas circunstancias resulta inútil.

- Lo indicado en el párrafo anterior no es aplicable a las órdenes INSERT, UPDATE, DELETE, ya que en estos casos no se levantan las excepciones correspondientes.

El siguiente ejemplo ilustra estas observaciones: se trata de un bloque que pretende cambiar la localidad de un departamento cuyo nombre en este caso es MARKETING (sabiendo que no existe ese departamento).

```
DECLARE
v_dpto depart.dnombre%TYPE := 'MARKETING'; --(NO existe)
v_loc depart.loc%TYPE;
BEGIN
UPDATE depart SET loc = 'SEVILLA'
WHERE dnombre = v_dpto; /* no actualiza ninguna fila
pero no levanta excepción */
IF SQL%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('Error en la actualización');
END IF;
DBMS_OUTPUT.PUT_LINE('Continúa el programa');
SELECT loc INTO v_loc
FROM depart
WHERE dnombre = v_dpto; /* fallará y levantará NO_DATA_FOUND */
IF SQL%NOTFOUND THEN
DBMS_OUTPUT.PUT_LINE('Nunca pasará por aquí');
END IF;
END;
```

El resultado de la ejecución del programa será:

Error en la actualización
Continúa el programa

...

ERROR at line 1:
ORA-01403: no data found
ORA-06512: at line

Cuando un SELECT...INTO hace referencia a una función de grupo nunca se levantará la excepción NO_DATA_FOUND, y SQL%FOUND siempre será verdadero. Esto se debe a que las funciones de grupo siempre retorna algún valor (aunque sea NULL o cero).

BEGIN

```
...
SELECT MAX(salario) INTO v_max
FROM emple
WHERE dept_no = num_depart; -- > nunca levantará
-- NO_DATA_FOUND
IF SQL%NOTFOUND THEN -- > nunca será cierto
... -- > nunca se ejecutará
END IF;
EXCEPTION
WHEN NO_DATA_FOUND THEN -- > no será invocado
...
```

Esta característica resulta útil para comprobar la existencia o no de una determinada fila sin que se levante NO_DATA_FOUND. Por ejemplo, para comprobar si existe un determinado departamento podemos escribir:

```
SELECT COUNT (*) INTO v_dummy
FROM depart WHERE dept_no = v_depart;
IF v_dummy > 0 THEN ...
```

3.10.8 Uso de cursores para actualizar filas

Los cursores se pueden usar para actualizar filas. En este caso, tenemos las siguientes opciones:

Cursor **FOR UPDATE**. Son cursores que permiten y facilitan la actualización (modificación o eliminación) de las filas seleccionadas por el cursor. Todas las filas seleccionadas serán bloqueadas tan pronto se abra el cursor (OPEN) y serán desbloqueadas al terminar las actualizaciones (al ejecutar COMMIT explícita o implícitamente).

Para crearlos únicamente habrá que añadirle FOR UPDATE al final de la declaración:

```
CURSOR <nombrercursor> IS <sentencia SELECT del cursor>
FOR UPDATE;
```

Los cursores así declarados se usan exactamente igual que los cursores explícitos estudiados (OPEN, FETCH, etcétera). Pero además de estas operaciones, permiten actualizar la última fila recuperada con FETCH mediante el comando

[illegible]

FOR UPDATE OF salario;

...

La utilización de la cláusula FOR UPDATE, en ocasiones, puede ser problemática pues:

- Se bloquean todas las filas de la SELECT, no sólo la que se está actualizando en un momento dado.
- Si se ejecuta un COMMIT, después ya no se puede ejecutar FETCH. Es decir, tenemos que esperar a que estén todas las filas actualizadas para confirmar los cambios.
- Uso de ROWID. Consiste en usar el identificador de fila (ROWID) como condición de:

- Al declarar el cursor en la cláusula SELECT, indicaremos que seleccione también el identificador de fila o ROWID:

CURSOR nombrecursor IS SELECT col1, col2, ..., ROWID FROM tabla;

- Al ejecutar el FETCH, se guardará el número de la fila en una variable o en un campo de la variable del cursor. Después ese número se usará en la cláusula WHERE de la actualización:

{UPDATE | DELETE} ... WHERE ROWID =
<variable_que_guarda_rowid>

En el ejemplo del epígrafe anterior:

```
CREATE OR REPLACE PROCEDURE subir_salario_dpto_b
(vp_num_dpto NUMBER,
vp_pct_subida NUMBER)
AS
CURSOR c_emple IS SELECT oficio, salario, ROWID
FROM emple WHERE dept_no = vp_num_dpto;
vc_reg_emple c_emple%ROWTYPE;
v_inc NUMBER(8,2);
BEGIN
OPEN c_emple;
FETCH c_emple INTO vc_reg_emple;
WHILE c_emple%FOUND LOOP
v_inc := (vc_reg_emple.salario / 100) * vp_pct_subida;
UPDATE emple SET salario = salario + v_inc
WHERE ROWID = vc_reg_emple.ROWID;
FETCH c_emple INTO vc_reg_emple;
END LOOP;
END subir_salario_dpto_b;
```

En caso de que tengamos que declarar explícitamente la variable que

recogerá el ROWID debemos tener en cuenta que esta pseudocolumna tiene su propio tipo con el mismo nombre (ROWID), tal como estudiamos en la unidad anterior. Por tanto, declararemos la variable así:

```
<nombredevariable> ROWID;
```

3.11 Excepciones

Las excepciones sirven para tratar errores en tiempo de ejecución, así como errores y situaciones definidas por el usuario.

Cuando se produce un error PL/SQL levanta una excepción y pasa el control a la sección EXCEPTION, donde buscará un manejador WHEN para la excepción o uno genérico (WHEN OTHERS) y dará por finalizada la ejecución del bloque actual.

El formato de la sección EXCEPTION es:

```
...  
EXCEPTION  
WHEN <NombredeExcepción1> THEN  
<instrucciones1>;  
WHEN <NombredeExcepción2> THEN  
<instrucciones2>;  
...  
[WHEN OTHERS THEN  
<instrucciones>;]  
END <nombre de programa>;
```

A continuación, estudiaremos los tres tipos de excepciones disponibles.

3.11.1 Excepciones internas predefinidas

Están predefinidas por Oracle. Se disparan automáticamente al producirse determinados errores. En la **tabla 3.11.1** adjunta se incluyen las excepciones más frecuentes con los códigos de error correspondientes.

No hay que declararlas en la sección DECLARE. Únicamente debemos incluir los manejadores WHEN con el tratamiento para cada excepción y/o un manejador genérico WHEN OTHERS que capturaré cualquier otra excepción.

Código error Oracle	Valor de SQL CODE	Excepción	Se disparan cuando...
ORA-06530	-6530	ACCESS_INTO_NULL	Se intenta acceder a los atributos de un objeto no inicializado.
ORA-06531	-6531	COLLECTION_IS_NULL	Se intenta acceder a elementos de una colección que no ha sido inicializada.
ORA-06511	-6511	CURSOR_ALREADY_OPEN	Intentamos abrir un cursor que ya se encuentra abierto.
ORA-00001	-1	DUP_VAL_ON_INDEX	Se intenta almacenar un valor que crearía duplicados en la clave primaria o en una columna con la restricción UNIQUE.
ORA-01001	-1001	INVALID_CURSOR	Se intenta realizar una operación no permitida sobre un cursor (por ejemplo, cerrar un cursor que no estaba abierto).
ORA-01722	-1722	INVALID_NUMBER	Fallo al intentar convertir una cadena a un valor numérico.
ORA-01017	-1017	LOGIN_DENIED	Se intenta conectar a ORACLE con un usuario o una clave no válidos.
ORA-01012	-1012	NOT_LOGGED_ON	Se intenta acceder a la base de datos sin estar conectado a Oracle.
ORA-01403	+100	NO_DATA_FOUND	Una sentencia SELECT ... INTO ... no devuelve ninguna fila.
ORA-06501	-6501	PROGRAM_ERROR	Hay un problema interno en la ejecución del programa.
ORA-06504	-6504	ROWTYPE_MISMATCH	La variable del cursor del HOST y la variable del cursor PL/SQL pertenecen a tipos incompatibles.
ORA-06533	-6533	SUBSCRIPT_OUTSIDE_LIMIT	Se intenta acceder a una tabla anidada o a un array con un valor de índice ilegal (por ejemplo, negativo).
ORA-06500	-6500	STORAGE_ERROR	El bloque PL/SQL se ejecuta fuera de memoria (o hay algún otro error de memoria).
ORA-00051	-51	TIMEOUT_ON_RESOURCE	Se excede el tiempo de espera para un recurso.
ORA-01422	-1422	TOO_MANY_ROWS	Una sentencia SELECT ... INTO ... devuelve más de una fila.
ORA-06502	-6502	VALUE_ERROR	Un error de tipo aritmético, de conversión, de truncamiento, etcétera.
ORA-01476	-1476	ZERO_DIVIDE	Se intenta la división entre cero.

Tabla 3.11.1 Excepciones más frecuentes en Oracle con los códigos de error.

DECLARE

...

BEGIN

...

EXCEPTION

WHEN NO_DATA_FOUND THEN

BDMS_OUTPUT.PUT_LINE ('ERROR datos no encontrados');

WHEN TOO_MANY_ROWS THEN

BDMS_OUTPUT.PUT_LINE ('ERROR demasiadas filas');

WHEN OTHERS THEN

BDMS_OUTPUT.PUT_LINE ('ERROR');

END;

3.11.2 Excepciones definidas por el usuario

Las excepciones definidas por el usuario se usan para tratar condiciones de error definidas por el programador.

Para su utilización hay que seguir tres pasos:

- 1 Se declaran en la sección DECLARE de la forma siguiente:
`<nombreexcepción> EXCEPTION;`
- 2 Se disparan o levantan en la sección ejecutable del programa con la orden

RAISE:

RAISE <nombreexcepción>;

- 3 Se tratan en la sección EXCEPTION según el formato ya conocido:

```
WHEN <nombreexcepción> THEN <tratamiento>;
DECLARE
...
importe_erroneo EXCEPTION;
...
BEGIN
...
IF precio NOT BETWEEN precio_min AND precio_maximo
THEN
RAISE importe_erroneo;
END IF;
...
EXCEPTION
...
WHEN importe_erroneo THEN
DBMS_OUTPUT.PUT_LINE('Importe erróneo. Venta cancelada.');
```

La instrucción RAISE se puede usar varias veces en el mismo bloque con la misma o con distintas excepciones, pero sólo puede haber un manejador WHEN para cada excepción.

```
DECLARE
...
venta_erronea EXCEPTION;
...
importe_erroneo EXCEPTION;
...
BEGIN
...
RAISE venta_erronea;
...
RAISE importe_erroneo;
...
RAISE venta_erronea;
...
EXCEPTION
...
WHEN importe_erroneo THEN
```

```
...;  
WHEN venta_erronea THEN  
...;  
END;
```

3.11.3 Otras Excepciones

Existen otros errores internos de Oracle que no tienen asignada una excepción. No obstante, generan (como cualquier otro error de Oracle) un código de error y un mensaje de error, a los que se accede mediante las funciones SQLCODE y SQLERRM.

Cuando se produce uno de estos errores también se transfiere el control a la sección EXCEPTION, donde se tratará el error en la cláusula WHEN OTHERS:

```
EXCEPTION  
...  
WHEN OTHERS THEN  
DBMS_OUTPUT.PUT_LINE('Error'||SQLCODE||SQLERRM);  
...  
END;
```

En este ejemplo, se muestra al usuario el texto 'ERROR' con el código de error y el mensaje de error utilizando las funciones correspondientes.

Estas dos funciones, SQLCODE y SQLERRM, no son accesibles desde órdenes SQL*Plus, pero se pueden pasar a este entorno a través de soluciones como la siguiente:

```
...  
WHEN OTHERS THEN  
:cod_err := SQLCODE;  
:msg_err := SQLERRM;  
ROLLBACK;  
EXIT;  
END;
```

Podemos asociar una excepción de usuario a alguno de estos errores internos que no tienen excepciones predefinidas asociadas. Para ello procederemos así:

- 1 Definimos la excepción en la sección de declaraciones como si fuese una excepción definida por el usuario:

```
<nombreexcepción> EXCEPTION;
```

[illegible]

con independencia del que la disparó.

- Si, después de tratar una excepción, queremos volver a la línea siguiente a la que se produjo, no podemos hacerlo directamente, pero sí es posible diseñar el programa para que funcione así. Esto se consigue encerrando el comando o comandos que pueden levantar la excepción en un bloque junto el tratamiento para la excepción:

```
SELECT INTO ... -- > Puede levantar NO_DATA_FOUND
```

```
...
```

Para que el control del programa no salga del bloque actual cuando se produzca una excepción, podemos encerrar el comando que puede levantar la excepción en un bloque y tratar la excepción en ese bloque:

```
...
```

```
BEGIN
```

```
SELECT INTO ... -- > Puede levantar NO_DATA_FOUND
```

```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
...; -- > tratamiento para la excepción
```

```
END;
```

- La cláusula WHEN OTHERS tratará cualquier excepción que no aparezca en las cláusulas WHEN anteriores, con independencia del tipo de excepción. De este modo, se evita que la excepción se propague a los bloques de nivel superior.

En el siguiente ejemplo, si se levanta la excepción ex1, se tratará en el mismo bloque, pero el control pasará después al bloque <<exterior>> en la línea siguiente a la llamada.

Si se hubiese levantado NO_DATA_FOUND, al no encontrar tratamiento, la excepción se propaga al bloque <<exterior>>, donde será tratada por WHEN OTHERS (suponiendo que no exista un tratamiento específico), devolviendo el control al bloque o la herramienta que llamó a <<exterior>>:

```
<<exterior>>
```

```
DECLARE
```

```
...
```

```
BEGIN
```

```
...
```

```
<<interior>>
```

```
DECLARE
```

```

...
ex1 EXCEPTION;
BEGIN

...
RAISE ex1;

...
SELECT col1, col2 INTO ...; --> Levanta NO_DATA_FOUND
...
EXCEPTION

...
WHEN ex1 THEN --> Tratamiento para ex1
ROLLBACK; --> Después pasará el control a (1)

...
END;
/*(1)*/...;

...
EXCEPTION

...
WHEN OTHERS THEN...
END;

```

- Si la excepción se levanta en la sección declarativa (por un fallo al inicializar una variable, por ejemplo), automáticamente se propagará al bloque que llamó al actual, sin comprobar si existe tratamiento para la excepción en el mismo bloque que se levantó.
- También se puede levantar una excepción en la sección EXCEPTION de forma voluntaria o por un error que se produzca al tratar una excepción. En este caso, también se propagará de forma automática al bloque que llamó al actual, sin comprobar si existe tratamiento para la excepción en el mismo bloque que se levantó. La excepción original (la que se estaba tratando cuando se produjo la nueva excepción) se perderá, ya que solamente puede estar activa una excepción.

```

EXCEPTION
WHEN INVALID_NUMBER THEN
INSERT INTO ...-- podría levantar DUP_VAL_ON_INDEX
WHEN DUP_VAL_ON_INDEX THEN -- no atraparé la exception

```

- En ocasiones, puede resultar conveniente volver a levantar la misma excepción después de tratarla para que se propague al bloque de nivel superior. Esto puede hacerse indicando al final de los comandos de manejo de la excepción el comando RAISE sin parámetros:

```

...

```


WHEN TOO_MANY_ROWS THEN

...; -- instrucciones de manejo del error

RAISE; -- levanta de nuevo la excepción y la propaga

- Las excepciones declaradas en un bloque son locales al bloque, no son conocidas en bloques de nivel superior. No se puede declarar dos veces la misma excepción en el mismo bloque, pero sí en distintos bloques. En este caso, la excepción del subbloque prevalecerá sobre la del bloque, aunque esta última se puede levantar desde el subbloque utilizando la notación de punto (RAISE *nombrebloque.nombreexcepción*).
- Las variables locales, las globales, los atributos de un cursor, y otros objetos del programa se pueden referenciar desde la sección EXCEPTION según las reglas de ámbito que rigen para los objetos del bloque. Pero si la excepción se ha disparado dentro de un bucle cursor FOR...LOOP, no se podrá acceder a los atributos del cursor, ya que Oracle cierra este cursor antes de disparar la excepción.
- En la sección EXCEPTION no se puede usar < GOTO etiqueta > para salir de esta sección o entrar en otra cláusula WHERE.
- Cuando no se encuentra tratamiento para una excepción en ninguno de los bloques o programas, Oracle da por finalizado con fallos el programa y ejecutará automáticamente un ROLLBACK deshaciendo todos los cambios pendientes de confirmación realizados por el programa.